PROJECT REPORT

Digital Design with Verilog HDL

DESIGN AND VERIFICATION OF A 4-BIT ARITHMETIC LOGIC UNIT (ALU)

supporting add, subtract, logic, shift, and sequential multiply/divide

Item	Details
Student	Lam Duy Loc
Student ID	SE190907
Design tool	Xilinx Vivado (Elaboration, Simulation, Synthesis)
Target device	xc7k325tffg900-2 (Kintex-7)
Language	Verilog HDL
Date	October 2026

TABLE OF CONTENTS

1. Introduction
2. Functional Specification
3. Design Architecture
4. Detailed Module Descriptions
5. Verification by Simulation
6. Synthesis and Resource Utilization
7. Analysis, Remarks, and Limitations
8. Conclusion

Appendix A. Module Source Code

1. Introduction

1.1. Project Objectives

This project designs a 4-bit Arithmetic Logic Unit (ALU) in the Verilog hardware description language using a hierarchical approach: each function is placed in its own module, and the modules are assembled in the top-level module `alu_4bit`. The ALU is the central block of a processor and performs arithmetic and logic operations on data.

The specific objectives are:

- Design a 4-bit adder/subtractor from basic logic gates (full adders connected as a ripple-carry chain).
- Design bitwise logic blocks (AND, OR, XOR, NOT) and a shifter block (SHL, SHR).
- Design a **sequential** multiplier and divider (shift-and-add / restoring division) controlled by a finite state machine (FSM), reusing the ALU's own adder/subtractor to save resources.
- Generate the four standard status flags (Carry, Zero, Overflow, Negative) and a divide-by-zero exception flag (`div_by_zero`).
- Build a **self-checking** testbench with a golden reference model and exhaustive coverage of all input combinations.
- Run behavioral simulation, elaboration, and synthesis in Vivado, and evaluate FPGA resource utilization.

1.2. Tools and Environment

Item	Details
EDA tool	Xilinx Vivado (Project Manager, RTL Analysis, Simulation, Synthesis)
Target device	xc7k325tffg900-2 – Kintex-7 family
Simulation	Vivado Simulator – Behavioral Simulation (Functional), top module: `alu_4bit_tb`
Simulation clock	10 ns period (100 MHz), `timescale 1ns / 1ps`
Language	Verilog (structural gate primitives plus behavioral `assign`/`always` descriptions)

1.3. Project Directory Structure

Table 1. Structure of the submitted project directory

Folder / file	Contents
`sources/`	9 RTL source files: `alu_4bit.v`, `adder_subtractor_4bit.v`, `full_adder.v`, `logic_unit_4bit.v`, `shifter_4bit.v`, `control_unit.v`, `muldiv_unit.v`, `result_mux.v`, `flag_unit.v`
`sim/alu_4bit_tb.v`	Self-checking testbench; prints student information, a demo table, and a PASS/FAIL summary

Folder / file	Contents
`Screenshots/F1…F6`	Evidence screenshots: source tree, waveform, Tcl Console, elaborated schematic, synthesized schematic, utilization report
`Screenshots/Truth_Table`	Truth table / instruction-set summary with simulation results for A = 0xD, B = 0xC

2. Functional Specification

2.1. Top-Level Port Interface

The top-level module `alu_4bit` has 15 data and control ports (30 I/O pins when bus widths are counted, matching Vivado's report of 30 I/O Ports).

Table 2. Port list of `alu_4bit`

Port	Dir.	Width	Description
`clk`	In	1	Clock; used only by the multiply/divide FSM
`rst`	In	1	Asynchronous active-high reset; clears all multiply/divide registers
`start`	In	1	Start pulse for multiply/divide; effective only when `opcode` is 1000 or 1001
`a`	In	4	Operand A (unsigned for MUL/DIV; two's complement when interpreting the overflow flag of ADD/SUB)
`b`	In	4	Operand B
`opcode`	In	4	Operation code
`result`	Out	8	Result. Single-cycle ops: lower 4 bits valid, upper 4 bits = 0. MUL/DIV use all 8 bits
`carry`	Out	1	Carry flag (C): carry-out of ADD/SUB, shifted-out bit of SHL/SHR
`zero`	Out	1	Zero flag (Z): 1 when all 8 bits of `result` are 0
`overflow`	Out	1	Signed overflow flag (V) of ADD/SUB
`negative`	Out	1	Negative flag (N): equals bit 3 of the result (disabled for MUL/DIV)
`div_by_zero`	Out	1	Divide-by-zero exception (meaningful only when `opcode` = DIV)
`busy`	Out	1	1 while the multiply/divide unit is computing
`done`	Out	1	1 when the multiply/divide has finished and the result is valid

2.2. Instruction Set

The ALU supports **10 operations**: 8 combinational operations that complete in a single cycle (the result is available after the combinational propagation delay) and 2 sequential operations (MUL, DIV) that complete after 5 clock cycles. Opcodes `1010`–`1111` are undefined and the ALU returns `result = 0`.

Table 3. ALU opcode table

Opcode	Name	Type / latency	Description	`result` format	Relevant flags
0000	ADD	Combinational	A + B	{0000, sum}	C, V, Z, N

Opcode	Name	Type / latency	Description	`result` format	Relevant flags
0001	SUB	Combinational	A − B (two's complement)	{0000, diff}	C, V, Z, N
0010	AND	Combinational	A & B	{0000, A&B}	Z, N
0011	OR	Combinational	A | B	{0000, A|B}	Z, N
0100	XOR	Combinational	A ^ B	{0000, A^B}	Z, N
0101	NOT	Combinational	~A (B unused)	{0000, ~A}	Z, N
0110	SHL	Combinational	Logical shift left by 1 bit	{0000, A<<1}	C (shifted-out MSB), Z, N
0111	SHR	Combinational	Logical shift right by 1 bit	{0000, A>>1}	C (shifted-out LSB), Z, N
1000	MUL	Sequential, 5 cycles	Unsigned A × B	{hi, lo} 8 bits	Z
1001	DIV	Sequential, 5 cycles	Unsigned A ÷ B	{remainder, quotient}	DBZ, Z
1010–1111	—	Undefined	Reserved	0x00	Z = 1

2.3. Status Flag Conventions

Table 4. Status flag equations (from `flag_unit.v`)

Flag	Logic expression	Meaning
Carry (C)	`(arith & c4) | (is_shl & a[3]) | (is_shr & a[0])`	ADD: carry-out of bit 3. SUB: 1 when there is no borrow (A ≥ B, ARM-style convention). SHL/SHR: the bit shifted out.
Overflow (V)	`arith & (c3 ^ c4)`	Signed two's-complement overflow: carry into the sign bit differs from carry out of it. Generated only for ADD/SUB.
Zero (Z)	`~|result`	Reduction NOR over all 8 result bits.
Negative (N)	`~is_muldiv & result[3]`	Sign bit of the 4-bit result; forced to 0 for MUL/DIV because their result is an unsigned 8-bit value.
Div_by_zero	`is_div & md_dbz`	Divide-by-zero exception, latched into a register at `start`.

2.4. Simulation Truth Table (A = 0xD, B = 0xC)

The table below (taken from the project's `Truth_Table` sheet) lists the simulated results for every opcode using the same inputs A = 1101_2 = 13 and B = 1100_2 = 12 (operand B is unused by NOT, SHL, and SHR). Each row was cross-checked against the RTL logic and matches.

Table 5. Simulation results with A = 13, B = 12

Opcode	Operation	Result (hex)	Explanation	Flags
0000	ADD	09	13 + 12 = 25 = 16 + 9 → 4-bit overflow, keeps 1001	C=1, Z=0, V=0, N=1
0001	SUB	01	13 − 12 = 1; no borrow so C = 1	C=1, Z=0, V=0, N=0

Opcode	Operation	Result (hex)	Explanation	Flags
0010	AND	0C	1101 & 1100 = 1100	Z=0, N=1
0011	OR	0D	1101 | 1100 = 1101	Z=0, N=1
0100	XOR	01	1101 ^ 1100 = 0001	Z=0, N=0
0101	NOT	02	~1101 = 0010	Z=0, N=0
0110	SHL	0A	1101 << 1 = 1010, shifted-out bit = 1	C=1, Z=0, N=1
0111	SHR	06	1101 >> 1 = 0110, shifted-out bit = 1	C=1, Z=0, N=0
1000	MUL	9C	13 × 12 = 156 = 0x9C (hi = 9, lo = C)	Z=0
1001	DIV	11	13 ÷ 12: remainder 1, quotient 1 → {0001, 0001}	DBZ=0, Z=0

3. Design Architecture

3.1. Overview

The design consists of 9 Verilog modules organized as a hierarchy with `alu_4bit` at the root. Inputs `a` and `b` are distributed in parallel to the functional blocks (adder/subtractor, logic unit, shifter, multiply/divide unit). All results are routed to `result_mux`, which selects one source according to `opcode`. Finally, `flag_unit` computes the status flags from the result and auxiliary signals (c3, c4, shifted-out bits).

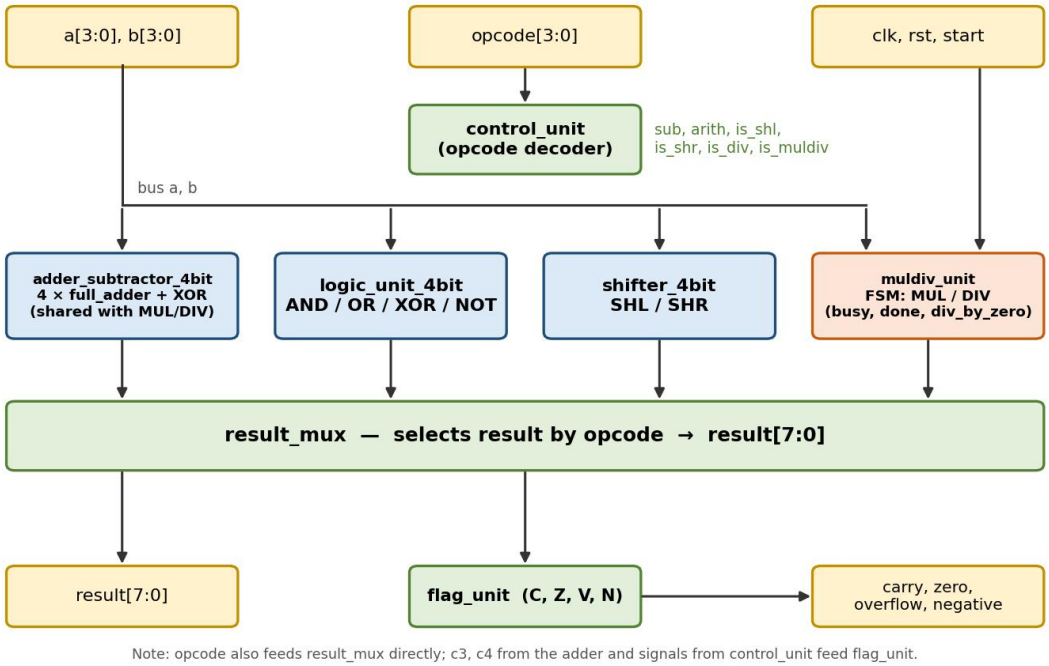


Figure 1. Overall architecture block diagram of `alu_4bit`

3.2. Module Hierarchy

The project source tree (Figure 2) shows that `alu_4bit` has 7 child instances. Among them, `u_addsub` contains 4 further `full_adder` instances (shown as "(4)" next to it).

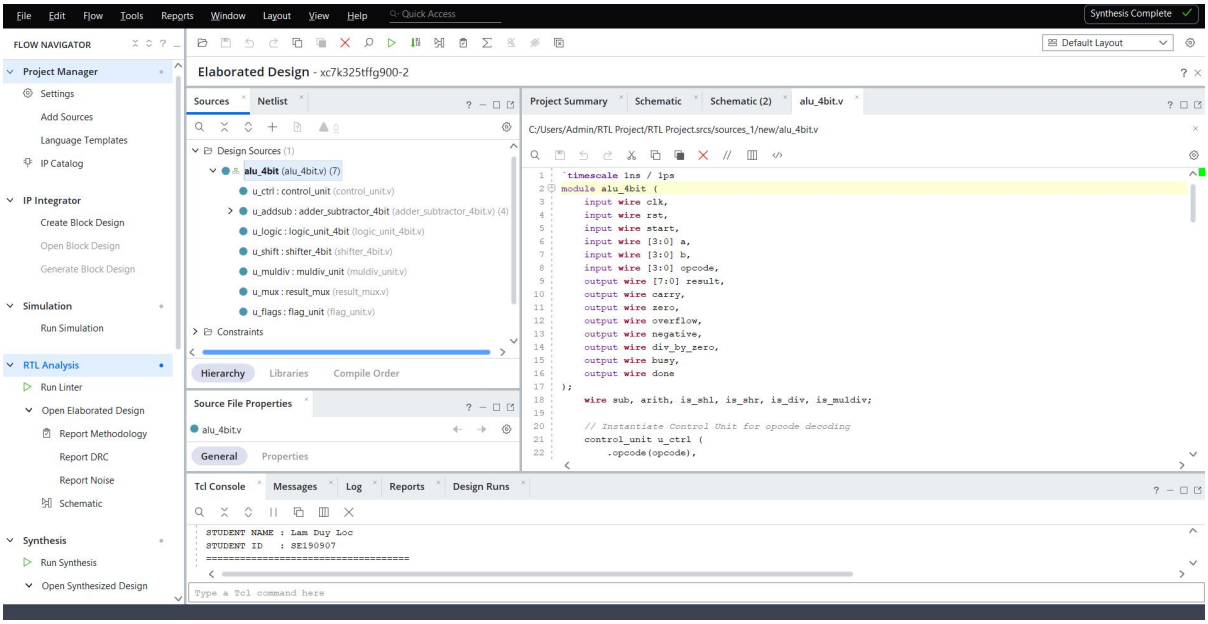


Figure 2. Source hierarchy and the `alu_4bit.v` source in Vivado

Table 6. Instances inside `alu_4bit`

Instance	Module	Circuit type	Role
`u_ctrl`	`control_unit`	Combinational	Decodes `opcode` into 6 internal control signals
`u_addsub`	`adder_subtractor_4bit`	Combinational (structural)	4-bit add/subtract; also outputs carries c3 and c4
`u_logic`	`logic_unit_4bit`	Combinational	AND, OR, XOR, NOT
`u_shift`	`shifter_4bit`	Combinational	Logical shift left/right by 1 bit; outputs the shifted-out bit
`u_muldiv`	`muldiv_unit`	Sequential (FSM)	Shift-and-add multiply, restoring divide; shares the adder
`u_mux`	`result_mux`	Combinational	Selects the 8-bit result by `opcode`
`u_flags`	`flag_unit`	Combinational	Generates C, Z, V, N
`fa0`…`fa3` (inside `u_addsub`)	`full_adder`	Combinational (gate-level)	1-bit full adder

3.3. Adder Sharing Between Combinational Operations and the Multiply/Divide Unit

The most notable design decision is the **shared adder/subtractor** used by both ADD/SUB and the multiply/divide unit. The top-level module contains three 2-to-1 multiplexers (shown as `RTL_MUX` in the schematic):

```
wire [3:0] add_in_a = busy ? md_add_a : a;
wire [3:0] add_in_b = busy ? md_add_b : b;
wire       add_sub  = busy ? md_add_sub : sub;
```

When `busy = 0`, the adder serves ADD/SUB directly from `a`, `b`, and `sub`. While the multiply/divide unit is running (`busy = 1`), the adder is handed to the FSM to perform the accumulating addition (multiply) or the trial subtraction (divide). As a result the entire ALU needs only **one 4-bit adder**, with no dedicated 4×4 multiplier or divider – saving area at the cost of waiting 5 cycles.

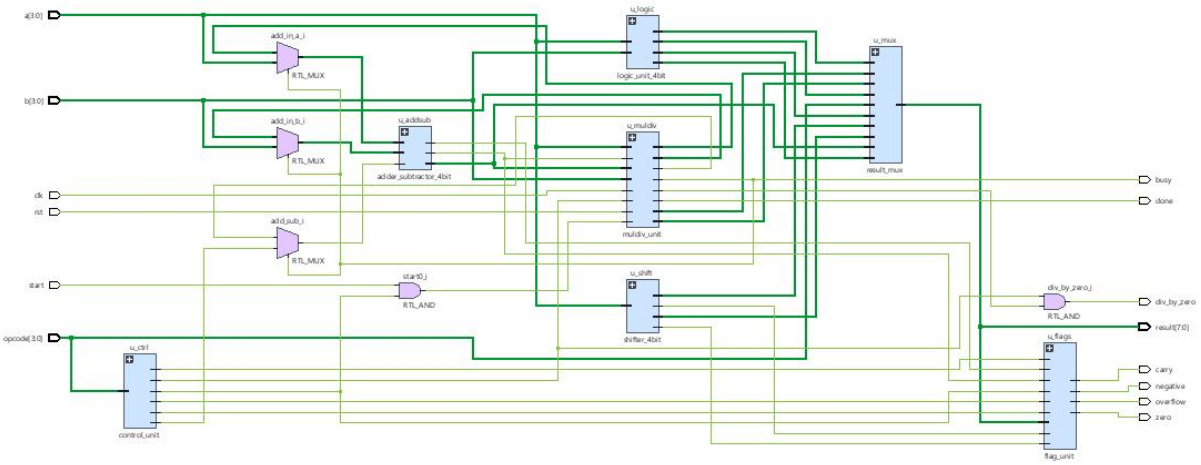


Figure 3. Elaborated schematic (RTL Analysis): three `RTL_MUX` cells sharing the adder, and `RTL_AND` cells forming `start` and `div_by_zero`

The two `RTL_AND` gates in the schematic correspond to `start & is_muldiv` (start is only issued when the opcode is MUL/DIV) and `is_div & md_dbz` (divide-by-zero is only reported while performing DIV).

4. Detailed Module Descriptions

4.1. `full_adder` – 1-bit Full Adder

Described at gate level using the `xor`, `and`, and `or` primitives, separating the **propagate** and **generate** signals:

- `prop = a ⊕ b` (propagate)
- `sum = prop ⊕ cin`
- `gen = a · b` (generate)
- `cout = gen + (prop · cin)`

Table 7. Full adder truth table

a	b	cin	sum	cout
0	0	0	0	0

a	b	cin	sum	cout
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

The structure uses 5 gates (2 XOR, 2 AND, 1 OR). The propagate/generate formulation makes a later upgrade to a carry-lookahead adder straightforward.

4.2. `adder_subtractor_4bit` – 4-bit Adder/Subtractor

Four `full_adder` instances are chained in a **ripple-carry** structure. Subtraction uses **two's complement**: A − B = A + (~B) + 1. The `sub` signal both inverts B (through four XOR gates) and is loaded as the initial carry-in `carry_chain[0]`:

- `sub = 0`: B passes through the XORs unchanged, cin = 0 → A + B.
- `sub = 1`: B is inverted, cin = 1 → A + ~B + 1 = A − B.

The module outputs two carry signals: `c3` (carry into bit 3) and `c4` (carry out of bit 3). These are used to compute the Carry and Overflow flags.

Example 13 + 12 (ADD) – computed bit by bit:

Table 8. Carry propagation when computing 1101 + 1100

Bit	a	b	cin	sum	cout
0	1	0	0	1	0
1	0	0	0	0	0
2	1	1	0	0	1
3	1	1	1	1	1

The result is sum = 1001 = 9, c3 = 1, c4 = 1. Therefore C = c4 = 1; V = c3 ⊕ c4 = 0; N = bit 3 = 1 – exactly as in the simulation table.

Example 13 − 12 (SUB): B = 1100 is inverted to 0011 with cin = 1. This gives sum = 0001 = 1, c3 = 1, c4 = 1 → C = 1 (no borrow), V = 0, N = 0 – matching the simulation table.

4.3. `logic_unit_4bit` – Bitwise Logic Unit

A purely combinational block that computes four results simultaneously using bitwise operators: `a & b`, `a | b`, `a ^ b`, and `~a`. Selection of the desired result is delegated to `result_mux`, so this block needs no control signal.

4.4. `shifter_4bit` – Shifter

- Logical shift left: `shl = {a[2:0], 1'b0}`; the shifted-out bit is `a[3]`.
- Logical shift right: `shr = {1'b0, a[3:1]}`; the shifted-out bit is `a[0]`.

The shifted-out bit is sent to `flag_unit` to form the Carry flag. These are **logical** shifts (zero fill), not arithmetic shifts.

4.5. `control_unit` – Opcode Decoder

Table 9. Control signals generated by `control_unit`

Signal	Condition	Effect
sub	`opcode == 0001`	Selects subtract mode of the adder/subtractor
arith	`opcode[3:1] == 000` (0000, 0001)	Enables arithmetic Carry and Overflow generation
is_shl	`opcode == 0110`	Enables Carry from the SHL shifted-out bit
is_shr	`opcode == 0111`	Enables Carry from the SHR shifted-out bit
is_div	`opcode == 1001`	Selects divide mode in the multiply/divide unit; divide-by-zero qualifier
is_muldiv	`opcode[3:1] == 100` (1000, 1001)	Enables `start`; disables the Negative flag

Comparing `opcode[3:1]` (ignoring the least-significant bit) recognizes a pair of adjacent opcodes (ADD/SUB or MUL/DIV) with a single 3-bit comparison.

4.6. `muldiv_unit` – Sequential Multiply/Divide Unit

This is the most complex module. It performs unsigned multiplication using the **shift-and-add** algorithm and unsigned division using the **restoring division** algorithm. Both run on the same set of registers and use the adder/subtractor supplied by `alu_4bit`.

4.6.1. Registers and Internal Signals

Table 10. Registers of `muldiv_unit` (18 flip-flops in total)

Register	Width	Role
acc	4	Accumulator: upper half of the product (multiply) / remainder (divide)
q	4	Multiply: holds the multiplier B and gradually becomes the lower half of the product. Divide: holds the dividend A and gradually becomes the quotient
m	4	Multiply: the multiplicand A. Divide: the divisor B
count	2	Counts the 4 iterations (0…3)
running	1	Running (drives `busy`)
mode_div	1	Latched mode: 0 = multiply, 1 = divide
done	1	Finished (drives `done`)
div_by_zero	1	Divide-by-zero flag latched at `start`

Sanity check: 4 + 4 + 4 + 2 + 1 + 1 + 1 + 1 = **18 flip-flops**, which matches the 18 Slice Registers in Vivado's utilization report (see Section 6).

4.6.2. Control State Machine

The FSM does not declare explicit states; instead it uses the `running` flag and the `count` counter. When `start = 1` while idle (`!running`), the unit loads its data (1 cycle); it then runs exactly 4 iterations; on the fourth (`count == 3`) it clears `running` and raises `done`. The total latency is **5 clock cycles** (1 load + 4 compute), exactly as the testbench verifies.

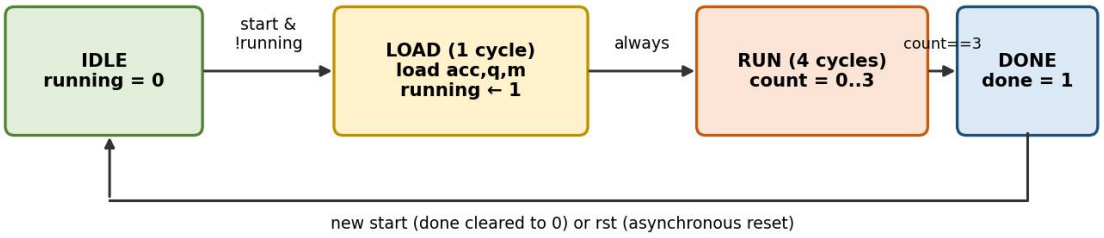


Figure 4. Control state diagram of `muldiv_unit`

The reset (`rst`) is asynchronous (`always @(posedge clk or posedge rst)`) and clears every register to 0 immediately.

4.6.3. Shift-and-Add Multiplication

On load: `acc = 0`, `q = B`, `m = A`. In each of the 4 iterations, bit `q[0]` is examined:

- If `q[0] = 1`: add `m` to `acc`, then shift the whole `{carry, acc, q}` group right by one bit: `{acc, q} <= {c4, sum, q[3:1]}`.
- If `q[0] = 0`: only shift right: `{acc, q} <= {1'b0, acc, q[3:1]}`.

After 4 iterations, `{acc, q}` holds the 8-bit product; `hi = acc`, `lo = q`. The adder's carry `c4` is shifted into the MSB of `acc`, so no bit is lost when the sum exceeds 4 bits.

Table 11. Execution trace of 13 × 12 – expected product 156 = 0x9C

Cycle	Action	acc	q	m
Load	acc=0, q=B, m=A	0000	1100	1101
1	q[0]=0 → shift right only	0000	0110	1101
2	q[0]=0 → shift right only	0000	0011	1101
3	q[0]=1 → acc + m = 1101, shift right	0110	1001	1101
4	q[0]=1 → acc + m = 1 0011, shift right	1001	1100	1101

Final result: `{acc, q} = 1001 1100 = 0x9C = 156` – identical to the simulated truth table. (This trace was reproduced with a Python model that follows the RTL logic.)

4.6.4. Restoring Division

On load: `acc = 0` (remainder), `q = A` (dividend), `m = B` (divisor). In each iteration:

1. Shift the `{acc, q}` pair left by one bit: the partial remainder is `r_shift = {acc[2:0], q[3]}`; the bit shifted out is `r_top = acc[3]`.
2. Trial subtraction: `add_sum = r_shift - m` (adder in `sub = 1` mode).
3. Condition `fits = r_top | add_c4`: `c4 = 1` means the subtraction did not borrow (r_shift ≥ m); `r_top = 1` means the true remainder is already ≥ 16 and therefore certainly ≥ m.
4. If `fits`: `acc <= add_sum`, `q <= {q[2:0], 1}` (record a quotient bit of 1). Otherwise **restore**: `acc <= r_shift`, `q <= {q[2:0], 0}`.

After 4 iterations, `acc` is the remainder and `q` is the quotient → `result = {remainder, quotient}`.

Table 12. Execution trace of 13 ÷ 12 – expected quotient 1, remainder 1 (0x11)

Cycle	Action	acc	q	m
Load	acc=0, q=A, m=B	0000	1101	1100
1	r_shift=0001 < 12 → restore, quotient bit 0	0001	1010	1100
2	r_shift=0011 < 12 → restore, quotient bit 0	0011	0100	1100
3	r_shift=0110 < 12 → restore, quotient bit 0	0110	1000	1100
4	r_shift=1101 ≥ 12 → acc = 13−12 = 1, quotient bit 1	0001	0001	1100

The final `{0001, 0001} = 0x11`: quotient = 1, remainder = 1 – matching the truth table.

4.6.5. Divide-by-Zero Handling

At `start`, the unit latches `div_by_zero <= is_div && (b == 0)`, and the top level ANDs the signal with `is_div`. Interestingly, the algorithm **needs no special branch** for this case: when `m = 0`, the trial subtraction is A + ~0 + 1 = A + 16, which always produces `c4 = 1`, so `fits` is true in all 4 iterations. Consequently the quotient is `1111` (0xF) and the remainder equals A. For example, 13 ÷ 0 gives `result = {1101, 1111} = 0xDF` together with `div_by_zero = 1` – exactly what the testbench expects.

I ran a cycle-accurate Python model of this unit over all 256 (A, B) pairs for both MUL and DIV: there is no mismatch against standard integer arithmetic (`a*b`, `a//b`, `a%b`, and the convention quotient = 15, remainder = A when dividing by zero).

4.7. `result_mux` – Result Selector

An `always @(*)` block uses a `case (opcode)` statement to place one source onto the `result[7:0]` bus:

Table 13. Selection table of `result_mux`

Opcode	Source	8-bit result
0000, 0001	`sum` (adder/subtractor)	{4'b0000, sum}
0010	`and_out`	{4'b0000, and_out}

Opcode	Source	8-bit result
0011	`or_out`	{4'b0000, or_out}
0100	`xor_out`	{4'b0000, xor_out}
0101	`not_out`	{4'b0000, not_out}
0110	`shl`	{4'b0000, shl}
0111	`shr`	{4'b0000, shr}
1000, 1001	`md_hi`, `md_lo`	{md_hi, md_lo}
Others (default)	—	8'b0000_0000

The explicit `default` branch assigning 0 prevents unwanted latch inference during synthesis.

4.8. `flag_unit` – Status Flag Unit

A purely combinational block implementing the four expressions given in Table 4. Points worth noting:

- **Carry** and **Overflow** are masked by `arith`, `is_shl`, and `is_shr`, so logic operations always give C = 0, V = 0.
- **Zero** examines all 8 bits of `result`, so for MUL/DIV it reflects the full 8-bit result.
- **Negative** takes `result[3]`, so for SHR (whose bit 3 is always 0) N is always 0; for MUL/DIV it is forced to 0 by `~is_muldiv`, because bit 3 of an 8-bit result is no longer a sign bit.

4.9. `alu_4bit` – Top-Level Module

The top-level module only does **interconnection**: it declares wires, creates the 3 adder-sharing multiplexers (Section 3.3), forms `start & is_muldiv` for the multiply/divide unit, forms `div_by_zero = is_div & md_dbz`, and wires up the 7 child modules. This style keeps each module small, readable, individually testable, and easy to replace (for example, swapping ripple-carry for carry-lookahead without affecting the rest).

5. Verification by Simulation

5.1. Verification Strategy

The testbench `alu_4bit_tb.v` follows a **self-checking** methodology in three parts: (1) printing student information and a visual demo table; (2) exhaustive checking of the combinational operations against a reference model; (3) exhaustive checking of MUL/DIV, including latency and the divide-by-zero exception.

Table 14. Test scale of the testbench

Part	Scope	Cases	Pass criterion
Demo	10 selected cases (ADD, SUB, AND, OR, XOR, NOT, SHL, SHR)	10	Visual inspection on the Tcl Console

Part	Scope	Cases	Pass criterion
Part 1	14 combinational opcodes (0–7 and 10–15) × 256 (A, B) pairs	3,584	`result`, C, V, Z, N match the golden model
Part 2 – MUL	256 (A, B) pairs, opcode 1000	256	`result == A×B`, latency exactly 5 cycles, Z correct
Part 2 – DIV	256 (A, B) pairs, opcode 1001	256	Quotient/remainder correct; B = 0 → `div_by_zero = 1`, quotient = F, remainder = A
Total		4,096 (excluding demo)	`err_count == 0` → prints "PASS"

5.2. Reference Model and Helper Tasks

- **Clock**: `always #5 sim_clk = ~sim_clk;` → 10 ns period (100 MHz).
- **Task `model`**: a golden model that computes the expected result and flags for the combinational operations, using a 5-bit register `adder_temp` for ADD/SUB (completely independent of the DUT's gate-level structure, so it is a meaningful cross-check).
- **Task `run_md`**: applies the opcode and operands on the falling clock edge, issues a one-cycle `start` pulse, then waits for `alu_done` while counting clock cycles to check latency.
- Changing signals on the **falling edge** (`negedge`) avoids race conditions with the DUT's rising edge.
- Comparisons use `!==` (case inequality), so `x`/`z` values are also detected – stricter than `!=`.

5.3. Demo Table Results on the Tcl Console

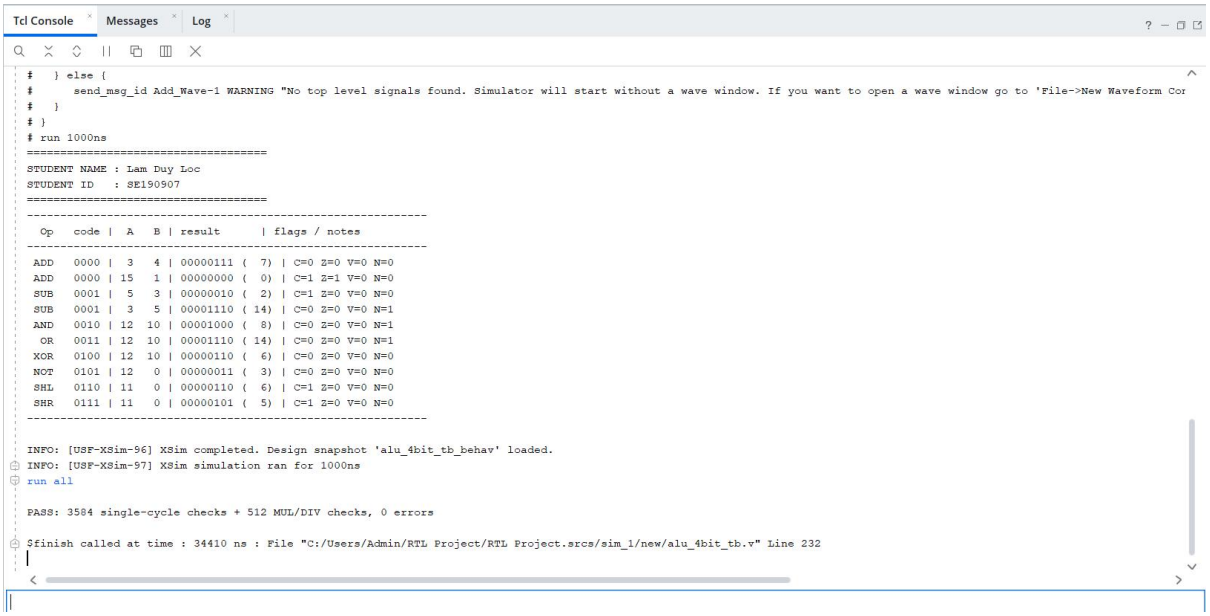


Figure 5. Tcl Console: student information and the 10-case demo table

Table 15. Cross-check of the demo table against hand calculation

Op	A	B	result	C Z V N	Hand check
ADD	3	4	0000 0111 (7)	0 0 0 0	3 + 4 = 7 ✔
ADD	15	1	0000 0000 (0)	1 1 0 0	15 + 1 = 16 → wraps to 0, carry = 1, Z = 1 ✔
SUB	5	3	0000 0010 (2)	1 0 0 0	5 − 3 = 2, no borrow so C = 1 ✔
SUB	3	5	0000 1110 (14)	0 0 0 1	3 − 5 = −2 = 1110 (two's complement), borrow so C = 0, N = 1 ✔
AND	12	10	0000 1000 (8)	0 0 0 1	1100 & 1010 = 1000, bit 3 = 1 → N = 1 ✔
OR	12	10	0000 1110 (14)	0 0 0 1	1100 | 1010 = 1110 ✔
XOR	12	10	0000 0110 (6)	0 0 0 0	1100 ^ 1010 = 0110 ✔
NOT	12	—	0000 0011 (3)	0 0 0 0	~1100 = 0011 ✔
SHL	11	—	0000 0110 (6)	1 0 0 0	1011 << 1 = 0110, shifted-out bit 1 → C = 1 ✔
SHR	11	—	0000 0101 (5)	1 0 0 0	1011 >> 1 = 0101, shifted-out bit 1 → C = 1 ✔

All 10 rows agree with the manual calculation, including the edge cases: 4-bit wrap-around (15 + 1), a negative result (3 − 5), and the carry/borrow flag of subtraction.

5.4. Waveform Analysis

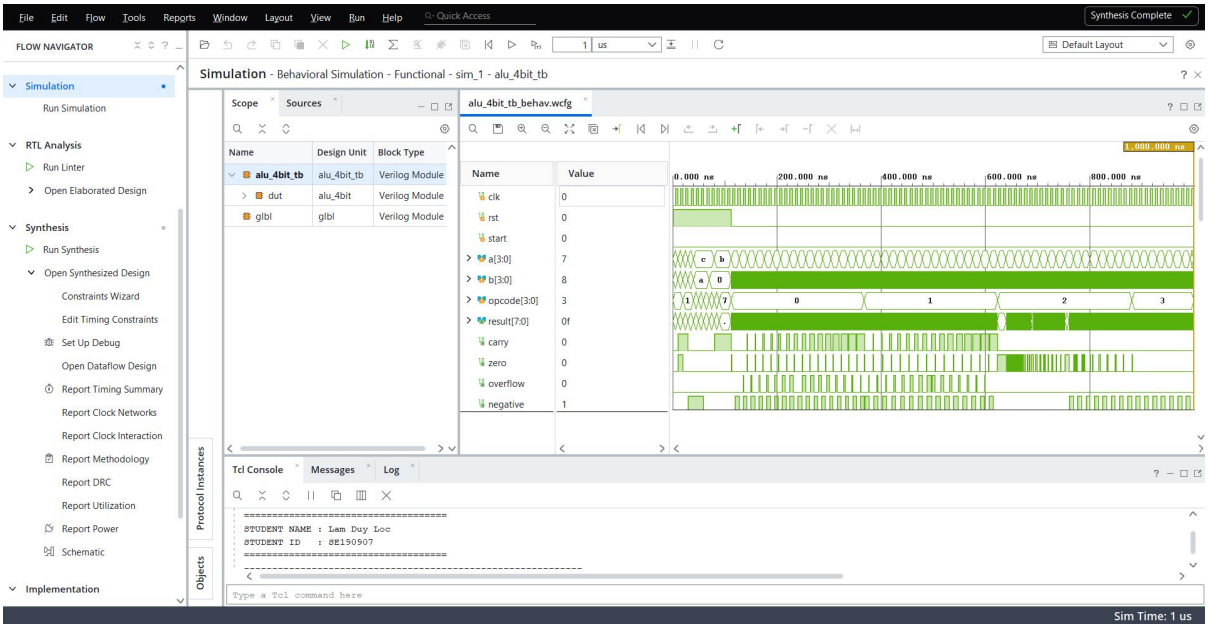


Figure 6. Behavioral simulation waveform of `alu_4bit_tb` (0 – 1000 ns)

Observations from the waveform, cross-checked against the testbench code:

- **Reset**: `rst` is high from 0 to about 112 ns and then goes low – consistent with the testbench (10 demo cases × 10 ns = 100 ns, plus `#12`).
- **Part 1 – exhaustive sweep**: `opcode` is held for segments of 256 ns each (each segment sweeps 256 A/B pairs at 1 ns each); opcodes 0, 1, 2, 3 appear in sequence (ADD → SUB → AND → OR). The boundaries lie near 368 ns, 624 ns, and 880 ns – matching 112 + 256 × k.
- **Spot check at 1 µs**: `opcode` = 3 (OR), `a` = 7, `b` = 8 → `result` = 0x0F (7 | 8 = 15), `negative` = 1 (bit 3 = 1), `carry` = `overflow` = `zero` = 0. The pair (7, 8) corresponds to index 0x78 = 120 = 1000 − 880 ns, exactly the sweep order in the testbench.
- `clk` toggles regularly every 10 ns; `start` and the multiply/divide signals are not yet active in the first microsecond because the MUL/DIV part runs after Part 1 finishes.

> **Note**: the waveform window shows only the first 1 µs, while the whole testbench runs much longer (Part 1 ≈ 14 × 256 ns; Part 2 ≈ 512 operations × 5–6 cycles × 10 ns). To fully document the multiply/divide behavior, add a waveform capture in which `start`, `busy`, and `done` are active.

5.5. Overall Result

The testbench prints "PASS: 3584 single-cycle checks + 512 MUL/DIV checks, 0 errors" when no mismatch occurs. The Tcl Console screenshot in the project shows only the beginning of the output (student information and the demo table) and does not include the final summary line.

> **Suggested additional evidence**: after the run completes, scroll the Tcl Console to the end and capture the "PASS … 0 errors" line to complete the verification evidence. In this report the

correctness of multiply/divide is further supported by a Python model reproducing the RTL algorithm (256/256 pairs match), but that does not replace the real Vivado simulation result.

6. Synthesis and Resource Utilization

6.1. Elaboration (RTL Analysis)

After Elaboration, Vivado reports **12 cells, 30 I/O ports, and 96 nets**. The cell count is small because the child modules (`u_ctrl`, `u_addsub`, `u_logic`, `u_shift`, `u_muldiv`, `u_mux`, `u_flags`) and the multiplexers/AND gates at the top level are still kept as block-level cells. The Design Runs table at the bottom of the screenshot records `synth_1` as complete with **60 LUTs and 18 FFs**, no BRAM, URAM, or DSP; `impl_1` has not been run.

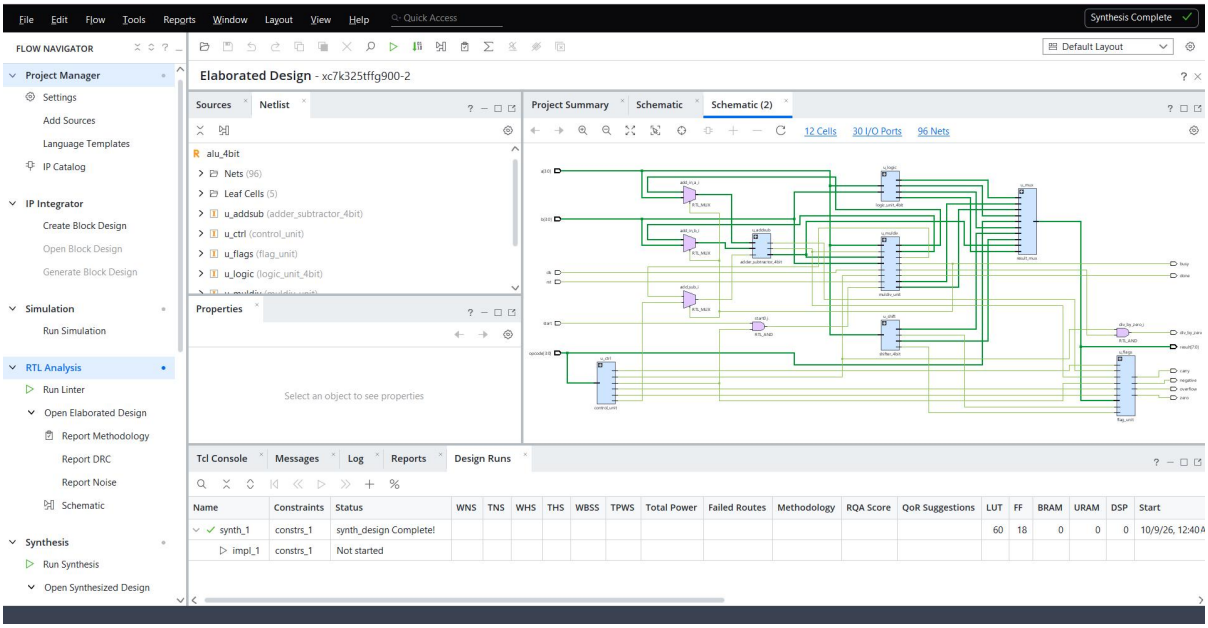


Figure 7. Elaborated design (12 cells, 30 I/O ports, 96 nets) and the Design Runs tab

6.2. Synthesis Results

After Synthesis, the netlist has **32 cells, 30 I/O ports, and 61 nets** with 31 leaf cells. Breakdown: 15 input buffers `IBUF` (`a`×4, `b`×4, `opcode`×4, `rst`, `start`, `clk`) + 1 `BUFG` for `clk` + 15 output buffers `OBUF` (8 `result` bits and 7 flag/status outputs) = 31 leaf cells, plus 1 hierarchical cell `u_muldiv` = 32 cells.

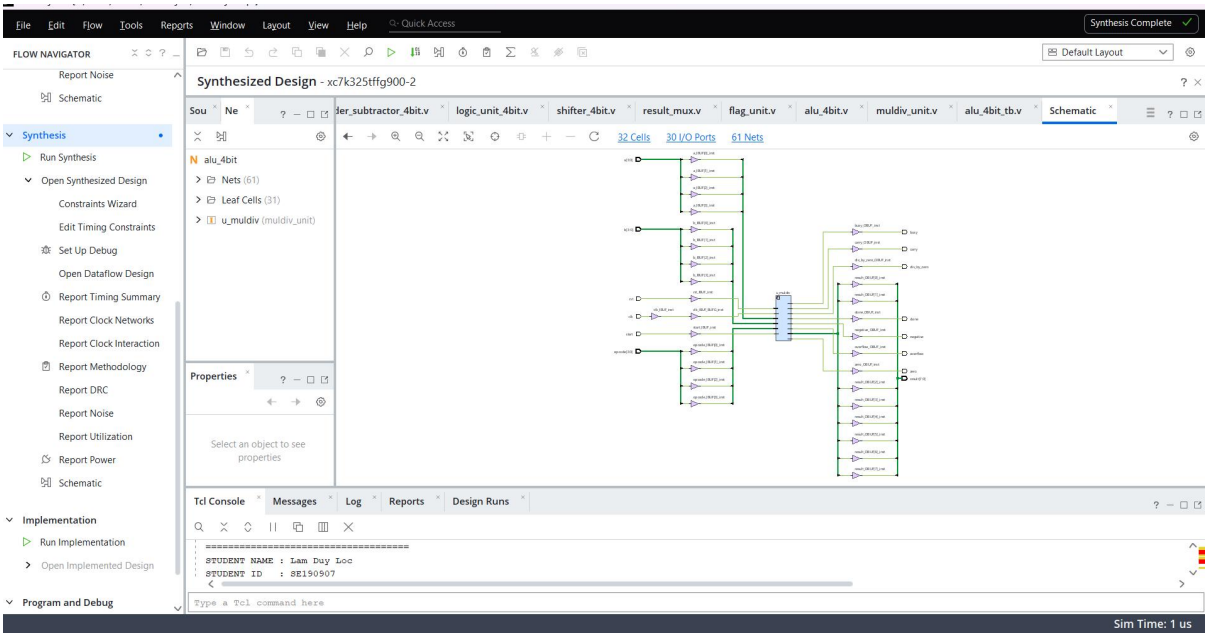


Figure 8. Schematic after Synthesis (32 cells, 30 I/O ports, 61 nets)

Notably, all the actual logic (adder, logic unit, shifter, result mux, flags) no longer appears as separate blocks at the top level; it has been optimized and merged into the `u_muldiv` cell. This is most likely the result of Vivado's cross-boundary optimization and hierarchy rebuilding, since these blocks are tightly coupled to the multiply/divide unit (in particular through the shared adder). This is normal behavior in synthesis and does not affect function.

6.3. Utilization Report

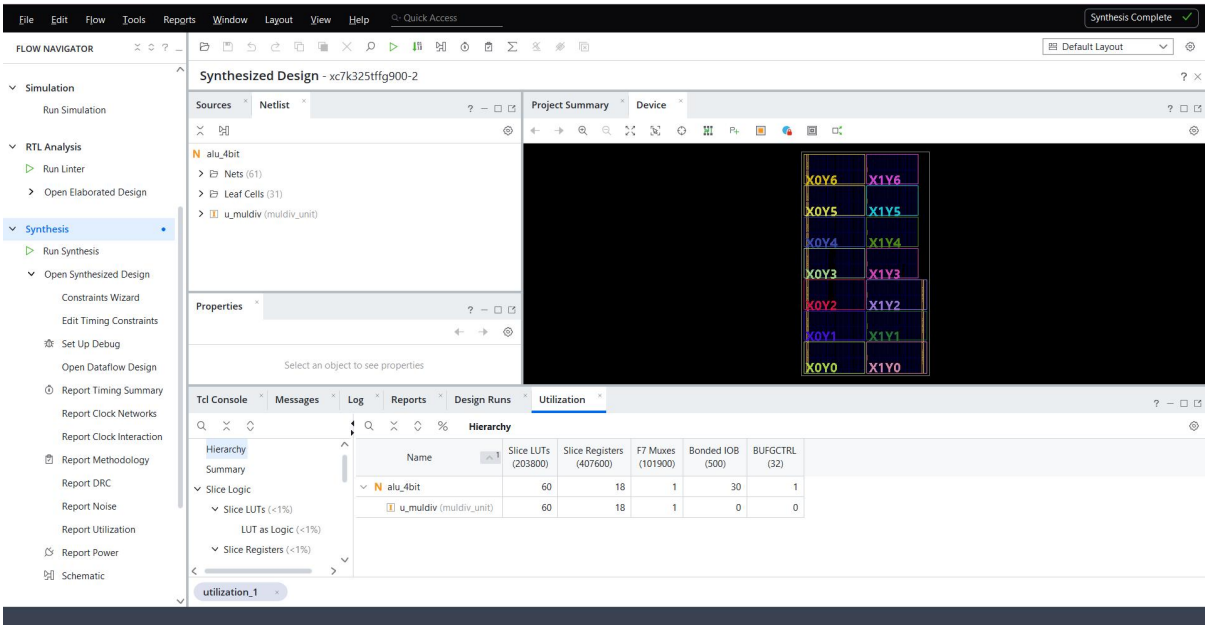


Figure 9. Utilization report (hierarchy view) after Synthesis

Table 16. Resource usage on xc7k325tffg900-2

Resource	Used	Available	Utilization
Slice LUTs	60	203,800	≈ 0.03 % (< 1 %)

Resource	Used	Available	Utilization
Slice Registers (FF)	18	407,600	≈ 0.004 % (< 1 %)
F7 Muxes	1	101,900	≈ 0.001 %
Bonded IOB	30	500	6.0 %
BUFGCTRL	1	32	3.1 %
BRAM / URAM / DSP	0 / 0 / 0	—	0 %

Remarks:

- The design is very compact: only 60 LUTs and 18 FFs, under 0.03 % of the LUTs of the Kintex-7 XC7K325T.
- No DSP48 is used for multiplication: it is implemented with the sequential shift-and-add algorithm on the shared adder, in line with the project's resource-saving goal.
- The relatively largest resource is I/O pins (30/500 = 6 %) because the ALU has wide buses; this is negligible when the ALU is embedded as a block inside a larger design.
- There is exactly 1 F7 Mux, indicating that part of the result/flag selection logic exceeds a single LUT6 and is combined with an F7 multiplexer.
- No timing constraints (XDC) exist yet, so the WNS/TNS columns in Design Runs are empty; maximum operating frequency cannot be evaluated.

7. Analysis, Remarks, and Limitations

7.1. Strengths of the Design

- **Clear hierarchy**: 9 small modules, each with a single task, easy to read and test individually.
- **Resource saving**: a single 4-bit adder serves ADD/SUB, MUL, and DIV.
- **Compact exception handling**: divide-by-zero is detected with a single flag and gives a deterministic result (quotient = 15, remainder = A) without a dedicated logic branch.
- **Strong verification**: 100 % of the input space is covered exhaustively (256 A/B pairs for every combinational opcode, MUL, and DIV), with an independent reference model and latency checking.
- **Mixed description styles**: both structural gate-level (`full_adder`, the XOR that inverts B) and behavioral (`result_mux`, `muldiv_unit`) descriptions, showing command of Verilog.

7.2. Limitations and Usage Notes

Table 17. Limitations observed from the source code

#	Issue	Explanation / consequence

#	Issue	Explanation / consequence
1	ADD/SUB cannot be performed while MUL/DIV is running	The adder is handed to the FSM when `busy = 1`; if a combinational ADD/SUB is requested at the same time, the adder output no longer reflects the current `a`, `b`. Keep `busy = 0` when using combinational operations.
2	The MUL/DIV `result` is valid only when `done = 1`	`result_mux` reads the `acc` and `q` registers directly; while running they hold intermediate values. The Zero flag is also reliable only when `done = 1`.
3	`result` depends on the current `opcode`	If `opcode` changes after `done`, `result_mux` switches source even though the multiply/divide registers still hold the result. Keep `opcode` stable until the result has been read.
4	`done` stays at 1 until the next `start`	There is no one-cycle `done` pulse; the consumer must do its own edge detection to catch the completion event.
5	SUB's Carry flag uses the "no borrow" convention	C = 1 when A ≥ B (ARM style), opposite to the "borrow" convention of some other processors (x86). Document this when integrating.
6	MUL/DIV support unsigned numbers only	There is no signed multiply/divide; the Negative flag is disabled for these two operations.
7	Opcodes 1010–1111 are undefined	They return `result = 0` and Z = 1 (no error is reported). They can be used for future extension.
8	No XDC constraints and Implementation not yet run	No timing (WNS/TNS), power, or maximum-frequency data is available.
9	PASS evidence is not in the screenshots	The Tcl Console screenshot does not show the testbench's final summary line.

7.3. Future Work

- Add arithmetic shift (ASR), rotate (ROL/ROR), and compare (CMP) operations, extending the opcode table.
- Upgrade the adder to **carry-lookahead** (the propagate/generate signals already exist) to reduce combinational delay.
- Implement signed multiplication with the **Booth** algorithm, or radix-4 multiply/divide, to reduce the cycle count.
- Write an XDC constraints file, run Implementation, report timing/power, and try the design on an FPGA board with switches/LEDs/7-segment displays.
- Add a one-cycle `done` pulse and a mechanism to latch `opcode`/results for a more robust interface.

8. Conclusion

The project delivers a 4-bit ALU with 10 operations – add, subtract, four logic operations, two shifts, and sequential multiply and divide – together with complete status flags and a divide-by-zero exception. The design is decomposed into 9 well-structured Verilog modules and reuses a single adder for both the combinational operations and the sequential multiply/divide unit.

For verification, the self-checking testbench sweeps 4,096 vectors exhaustively (3,584 combinational operations and 512 multiply/divide operations) against a reference model, including the 5-cycle latency and the divide-by-zero exception. The demo table and the waveform in Vivado agree with the theoretical analysis.

For implementation, synthesis on the Kintex-7 xc7k325tffg900-2 shows a very compact design: **60 LUTs, 18 FFs, 1 F7 Mux, 0 BRAM, 0 DSP**, a negligible fraction of the chip. The remaining limitations (timing constraints, implementation, the final PASS evidence) are listed in Section 7 as directions for further completion.

Appendix A. Module Source Code

Complete source code of the 9 modules in the `sources/` folder (original content unchanged).

alu_4bit.v

```
`timescale 1ns / 1ps
module alu_4bit (
    input wire clk,
    input wire rst,
    input wire start,
    input wire [3:0] a,
    input wire [3:0] b,
    input wire [3:0] opcode,
    output wire [7:0] result,
    output wire carry,
    output wire zero,
    output wire overflow,
    output wire negative,
    output wire div_by_zero,
    output wire busy,
    output wire done
);
    wire sub, arith, is_shl, is_shr, is_div, is_muldiv;

    // Instantiate Control Unit for opcode decoding
    control_unit u_ctrl (
        .opcode(opcode),
        .sub(sub),
        .arith(arith),
        .is_shl(is_shl),
        .is_shr(is_shr),
        .is_div(is_div),
        .is_muldiv(is_muldiv)
    );

    wire [3:0] md_add_a, md_add_b;
    wire md_add_sub;
    wire [3:0] add_in_a = busy ? md_add_a : a;
    wire [3:0] add_in_b = busy ? md_add_b : b;
    wire       add_sub  = busy ? md_add_sub : sub;

    wire [3:0] sum;
    wire c3, c4;

    // Instantiate Adder/Subtractor unit
    adder_subtractor_4bit u_addsub (
        .a(add_in_a),
        .b(add_in_b),
        .sub(add_sub),
        .sum(sum),
        .c3(c3),
        .c4(c4)
    );

    wire [3:0] and_out, or_out, xor_out, not_out;
    wire [3:0] shl, shr;
    wire shl_out_bit, shr_out_bit;

    // Instantiate Logic Unit (AND, OR, XOR, NOT)
    logic_unit_4bit u_logic (
        .a(a),
        .b(b),
        .and_out(and_out),
        .or_out(or_out),
        .xor_out(xor_out),
        .not_out(not_out)
    );

    // Instantiate Shifter unit (SHL, SHR)
    shifter_4bit u_shift (
        .a(a),
        .shl(shl),
```

```
        .shr(shr),
        .shl_out_bit(shl_out_bit),
        .shr_out_bit(shr_out_bit)
    );

    wire [3:0] md_hi, md_lo;
    wire md_dbz;

    // Instantiate Sequential Multiplier & Divider unit
    muldiv_unit u_muldiv (
        .clk(clk),
        .rst(rst),
        .start(start & is_muldiv),
        .is_div(is_div),
        .a(a),
        .b(b),
        .add_a(md_add_a),
        .add_b(md_add_b),
        .add_sub(md_add_sub),
        .add_sum(sum),
        .add_c4(c4),
        .hi(md_hi),
        .lo(md_lo),
        .busy(busy),
        .done(done),
        .div_by_zero(md_dbz)
    );

    assign div_by_zero = is_div & md_dbz;

    // Instantiate Result Multiplexer to route final output
    result_mux u_mux (
        .opcode(opcode),
        .sum(sum),
        .and_out(and_out),
        .or_out(or_out),
        .xor_out(xor_out),
        .not_out(not_out),
        .shl(shl),
        .shr(shr),
        .md_hi(md_hi),
        .md_lo(md_lo),
        .result(result)
    );

    // Instantiate Flag Unit to compute status flags
    flag_unit u_flags (
        .result(result),
        .arith(arith),
        .is_shl(is_shl),
        .is_shr(is_shr),
        .is_muldiv(is_muldiv),
        .c3(c3),
        .c4(c4),
        .shl_out_bit(shl_out_bit),
        .shr_out_bit(shr_out_bit),
        .carry(carry),
        .zero(zero),
        .overflow(overflow),
        .negative(negative)
    );
endmodule
```

adder_subtractor_4bit.v

```
`timescale 1ns / 1ps
module adder_subtractor_4bit (
    input wire [3:0] a,
    input wire [3:0] b,
    input wire sub,
    output wire [3:0] sum,
    output wire c3,
    output wire c4
);
```

```
    wire [3:0] b_inverted;
    wire [4:0] carry_chain;


    assign carry_chain[0] = sub;


    // Invert operand B and add 1 for two's complement subtraction
    xor xor_b0 (b_inverted[0], b[0], sub);
    xor xor_b1 (b_inverted[1], b[1], sub);
    xor xor_b2 (b_inverted[2], b[2], sub);
    xor xor_b3 (b_inverted[3], b[3], sub);


    // 4-bit Ripple Carry Adder instantiation
    full_adder fa0 (.a(a[0]), .b(b_inverted[0]), .cin(carry_chain[0]), .sum(sum[0]), .cout(carry_chain[1]));
    full_adder fa1 (.a(a[1]), .b(b_inverted[1]), .cin(carry_chain[1]), .sum(sum[1]), .cout(carry_chain[2]));
    full_adder fa2 (.a(a[2]), .b(b_inverted[2]), .cin(carry_chain[2]), .sum(sum[2]), .cout(carry_chain[3]));
    full_adder fa3 (.a(a[3]), .b(b_inverted[3]), .cin(carry_chain[3]), .sum(sum[3]), .cout(carry_chain[4]));


    assign c3 = carry_chain[3];
    assign c4 = carry_chain[4];
endmodule
```

full_adder.v

```
`timescale 1ns / 1ps
module full_adder (
    input wire a,
    input wire b,
    input wire cin,
    output wire sum,
    output wire cout
);
    wire prop_sig;
    wire gen_sig;
    wire and_prop_cin;


    // 1-bit Full Adder logic: Compute sum and carry-out
    xor xor_prop (prop_sig, a, b);
    xor xor_sum  (sum, prop_sig, cin);
    and and_gen  (gen_sig, a, b);
    and and_pc   (and_prop_cin, prop_sig, cin);
    or  or_cout  (cout, gen_sig, and_prop_cin);
endmodule
```

logic_unit_4bit.v

```
`timescale 1ns / 1ps
module logic_unit_4bit (
    input wire [3:0] a,
    input wire [3:0] b,
    output wire [3:0] and_out,
    output wire [3:0] or_out,
    output wire [3:0] xor_out,
    output wire [3:0] not_out
);
    // Perform bitwise logical operations (AND, OR, XOR, NOT) based on control signal
    assign and_out = a & b;
    assign or_out  = a | b;
    assign xor_out = a ^ b;
    assign not_out = ~a;
endmodule
```

shifter_4bit.v

```
`timescale 1ns / 1ps
module shifter_4bit (
    input wire [3:0] a,
    output wire [3:0] shl,
    output wire [3:0] shr,
    output wire shl_out_bit,
```

```
    output wire shr_out_bit
);
    // Perform Logical Shift Left (SHL) and Logical Shift Right (SHR)
    assign shl         = {a[2:0], 1'b0};
    assign shr         = {1'b0, a[3:1]};


    // Capture the shifted-out bit (MSB or LSB) for the carry flag
    assign shl_out_bit = a[3];
    assign shr_out_bit = a[0];
endmodule
```

control_unit.v

```
`timescale 1ns / 1ps
module control_unit (
    input wire [3:0] opcode,
    output wire sub,
    output wire arith,
    output wire is_shl,
    output wire is_shr,
    output wire is_div,
    output wire is_muldiv
);
    // Decode 4-bit opcode into internal control signals for active modules
    assign sub      = (opcode == 4'b0001);
    assign arith    = (opcode[3:1] == 3'b000);
    assign is_shl   = (opcode == 4'b0110);
    assign is_shr   = (opcode == 4'b0111);
    assign is_div   = (opcode == 4'b1001);
    assign is_muldiv= (opcode[3:1] == 3'b100);
endmodule
```

muldiv_unit.v

```
// ======================================================================
// Module      : muldiv_unit
// Description : Sequential 4-bit unsigned multiplier (shift-and-add)
//               and restoring divider. Executes in 5 clock cycles.
// ======================================================================



`timescale 1ns / 1ps
module muldiv_unit (
    input wire clk,
    input wire rst,
    input wire start,
    input wire is_div,
    input wire [3:0] a,
    input wire [3:0] b,
    output wire [3:0] add_a,
    output wire [3:0] add_b,
    output wire add_sub,
    input wire [3:0] add_sum,
    input wire add_c4,
    output wire [3:0] hi,
    output wire [3:0] lo,
    output wire busy,
    output reg done,
    output reg div_by_zero
);
    reg [3:0] acc, q, m;
    reg [1:0] count;
    reg running;
    reg mode_div;


    wire [3:0] r_shift = {acc[2:0], q[3]};
    wire r_top = acc[3];
    wire fits = r_top | add_c4;


    assign add_a   = mode_div ? r_shift : acc;
    assign add_b   = m;
    assign add_sub = mode_div;
```

```
    assign busy    = running;
    assign hi      = acc;
    assign lo      = q;

    // FSM (Finite State Machine) logic for sequential operations
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            acc         <= 4'd0;
            q           <= 4'd0;
            m           <= 4'd0;
            count       <= 2'd0;
            running     <= 1'b0;
            mode_div    <= 1'b0;
            done        <= 1'b0;
            div_by_zero <= 1'b0;
        end else if (start && !running) begin
            running     <= 1'b1;
            done        <= 1'b0;
            mode_div    <= is_div;
            count       <= 2'd0;
            acc         <= 4'd0;
            q           <= is_div ? a : b;
            m           <= is_div ? b : a;

            // Exception handling: Detect and set flag for division by zero
            div_by_zero <= is_div && (b == 4'd0);
        end else if (running) begin
            if (!mode_div) begin
                // Datapath for shift-and-add multiplier algorithm
                if (q[0])
                    {acc, q} <= {add_c4, add_sum, q[3:1]};
                else
                    {acc, q} <= {1'b0, acc, q[3:1]};
            end else begin
                // Datapath for restoring division algorithm
                if (fits) begin
                    acc <= add_sum;
                    q   <= {q[2:0], 1'b1};
                end else begin
                    acc <= r_shift;
                    q   <= {q[2:0], 1'b0};
                end
            end
            count <= count + 2'd1;
            if (count == 2'd3) begin
                running <= 1'b0;
                done    <= 1'b1;
            end
        end
    end
endmodule
```

result_mux.v

```
`timescale 1ns / 1ps
module result_mux (
    input wire [3:0] opcode,
    input wire [3:0] sum,
    input wire [3:0] and_out,
    input wire [3:0] or_out,
    input wire [3:0] xor_out,
    input wire [3:0] not_out,
    input wire [3:0] shl,
    input wire [3:0] shr,
    input wire [3:0] md_hi,
    input wire [3:0] md_lo,
    output reg [7:0] result
);
    // Route the correct computed result to the final 8-bit output bus based on opcode
    always @(*) begin
        case (opcode)
            4'b0000, 4'b0001: result = {4'b0000, sum};
            4'b0010:          result = {4'b0000, and_out};
            4'b0011:          result = {4'b0000, or_out};
            4'b0100:          result = {4'b0000, xor_out};
            4'b0101:          result = {4'b0000, not_out};
```

```
            4'b0110:          result = {4'b0000, shl};
            4'b0111:          result = {4'b0000, shr};
            4'b1000, 4'b1001: result = {md_hi, md_lo};
            default:          result = 8'b0000_0000;
        endcase
    end
endmodule
```

flag_unit.v

```
`timescale 1ns / 1ps
// ======================================================================
// Module      : flag_unit
// Description : Generates ALU status flags (Carry, Zero, Overflow,
//               Negative, and Div_by_zero exception) based on operations.
// ======================================================================
module flag_unit (
    input wire [7:0] result,
    input wire arith,
    input wire is_shl,
    input wire is_shr,
    input wire is_muldiv,
    input wire c3,
    input wire c4,
    input wire shl_out_bit,
    input wire shr_out_bit,
    output wire carry,
    output wire zero,
    output wire overflow,
    output wire negative
);
    // Update status flags (Carry, Zero, Overflow, Negative) based on the active operation
    assign carry    = (arith & c4) | (is_shl & shl_out_bit) | (is_shr & shr_out_bit);
    assign overflow = arith & (c3 ^ c4);
    assign zero     = ~|result;
    assign negative = ~is_muldiv & result[3];
endmodule
```